
Global Well-Being & Human Development Index (HDI) Predictor Dashboard

Project Delivery Documentation

1. Project Title & Overview

Project Description

The Global Well-Being & Human Development Index (HDI) Predictor Dashboard is a lightweight, self-contained analytics web application designed to bring predictive intelligence to development-policy planning. Rather than relying on retrospective HDI publications alone, the platform lets a user supply a country's current socioeconomic indicators — such as education spending, life expectancy, GNI per capita, and access-to-services metrics — and instantly returns a data-driven estimate of the resulting well-being score.

At its core, the dashboard wraps a pre-trained multi-variable regression model in a minimal Flask service so that predictive analytics can be embedded directly into policy discussions instead of being confined to offline notebooks. This matters for resource mapping because planners rarely have the luxury of waiting on a full statistical office release cycle; the tool gives them a fast, repeatable way to sanity-check proposed budget shifts, compare regions on equal footing, and flag scenarios where a policy change would push a metric outside historically observed, and therefore reliable, bounds. By keeping the whole stack local — a flat serialized model file and a CSV of historical country metrics — the project avoids the operational overhead of a database or cloud deployment while still producing analytically defensible output.

User Scenarios

Scenario 1 — Regional Policy Researcher: Education Spend Sensitivity

A regional policy researcher opens the dashboard to test how a proposed 10% increase in education expenditure would move a country's predicted HDI. They enter the country's baseline indicator set into the Scenario Simulator, then submit a second run with the education-spend field increased by 10%, holding every other indicator constant. The dashboard returns both the baseline and adjusted predictions side by side, along with the delta and the percentage change, giving the researcher a defensible, quantified talking point for a funding proposal rather than a qualitative guess.

Scenario 2 — Data Science Academic: Historical Outlier Validation

A data science academic wants to confirm that the regression model behaves sensibly on known historical outliers before citing it in coursework. They pull a handful of rows from HDI.csv that correspond to countries with unusual indicator combinations (for example, high GNI per capita but comparatively low life expectancy), feed those exact rows through the Predictor Engine, and compare the model's output against the country's actual recorded HDI for that year. Because the comparison is done directly against the historical dataset the

model was validated on, the academic can quickly judge whether prediction error is within an acceptable range or whether the model is systematically biased for that class of country.

Scenario 3 — International Development Audit Team: Extreme Scenario Limits

An audit team evaluating a proposed resource-allocation plan wants to make sure nobody accidentally models a physically impossible or policy-irrelevant scenario — for instance, a literacy rate above 100% or a negative life-expectancy delta. Using the Scenario Simulator's clamped input ranges, the team deliberately pushes several indicators to their upper and lower bounds and confirms that the interface rejects or clamps out-of-range values with a clear message rather than silently returning a nonsensical prediction. This gives the audit team confidence that downstream users of the tool cannot misallocate resources based on a modeling artifact.

2. Technical Architecture & Component Map

Data Flow Diagram

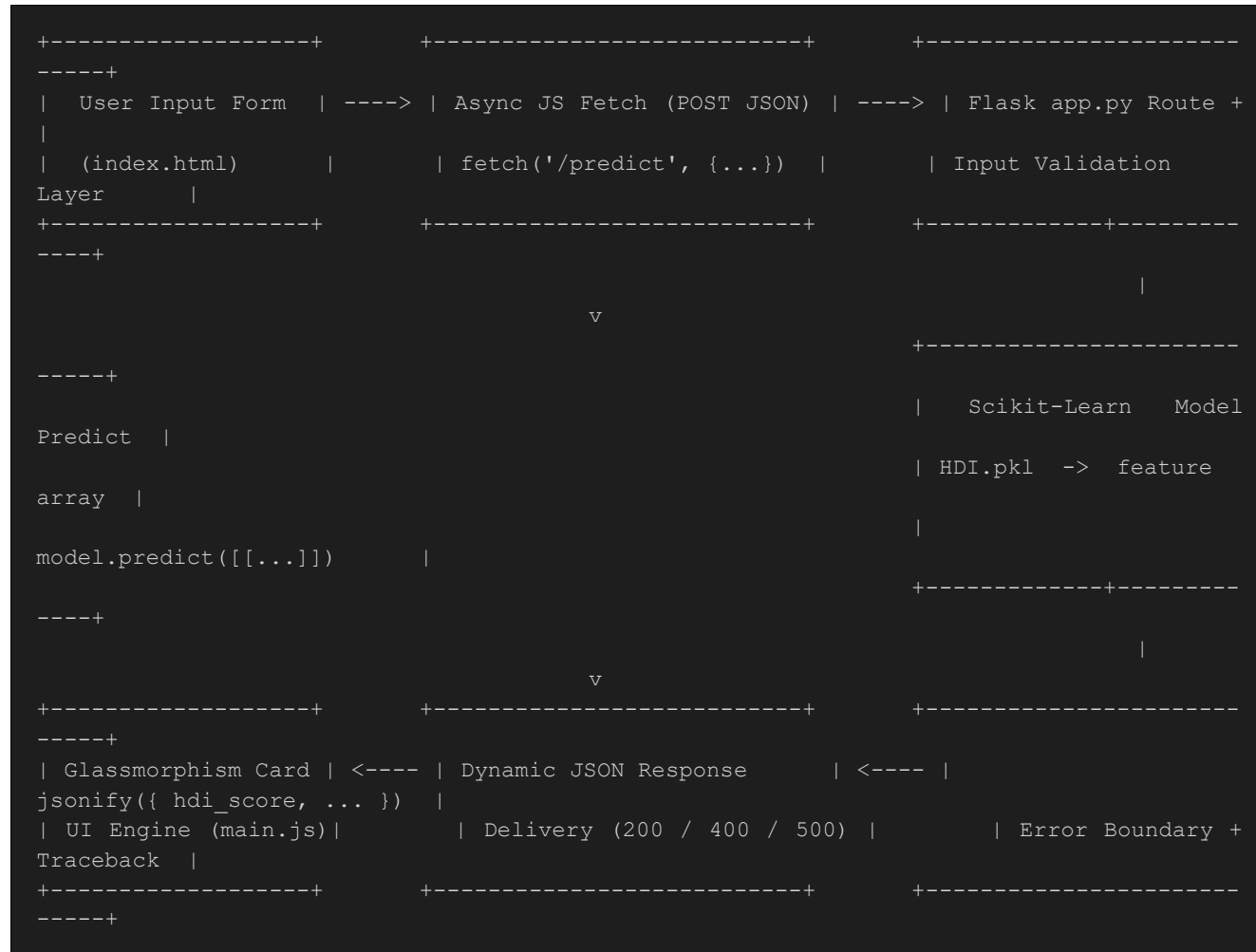


Figure 2.1 — End-to-end request lifecycle from form submission to rendered result card.

Architectural Description

The application follows a modular three-tier layout: a static HTML/CSS/JavaScript frontend, a single-process Flask backend, and a flat-file model/data layer. When a user submits the input form, client-side JavaScript intercepts the default form action, serializes the field values into a JSON payload, and sends it asynchronously to the Flask backend via the Fetch API. This keeps the page from reloading and lets the UI show a loading state while the prediction is computed.

On the server, app.py exposes dedicated routes for each of the three core modules — prediction, scenario simulation, and feature analysis. Each route performs input validation before touching the model: required fields are checked for presence, numeric fields are range-clamped against realistic socioeconomic bounds, and malformed payloads short-circuit to a 400 response with a descriptive error message. Once validated, the indicators are assembled into a single ordered NumPy array matching the exact feature order the model was

trained on, and passed to `model.predict()`. The resulting continuous well-being score, together with any derived comparison metrics, is serialized back to the client as JSON.

The project deliberately avoids a relational database. Because the model is a static, pre-trained artifact and the reference dataset (`HDI.csv`) is read-only historical data used only for validation lookups, both are loaded once into memory as NumPy/Pandas structures at process startup. This flat-file approach removes an entire class of deployment complexity — no schema migrations, no connection pooling, no query layer — while still giving the Feature Analysis Deck fast, in-memory access to the model's learned coefficients and to representative historical rows for comparison. The trade-off is intentional: the dataset is small enough, and access patterns simple enough (point lookups and one-shot predictions), that an in-memory array outperforms a database round trip and keeps the whole application deployable from a single directory.

3. Prerequisites & Development Setup Guide

Prerequisites

- **Python 3.9+:** Required for compatibility with the pinned Scikit-Learn and Flask versions.
- **pip:** Used for dependency installation from requirements.txt.
- **A modern browser:** Chrome, Edge, or Firefox for testing the Fetch-API-driven frontend.
- **Basic Scikit-Learn familiarity:** Useful for interpreting the regression model's coefficients in the Feature Analysis Deck.
- **Git:** For version control and to keep HDI.pkl and HDI.csv tracked (or intentionally .gitignored, depending on size).
- **Ngrok account (optional):** Only required for the public tunnel deployment step described in Milestone 5.

Environment Setup

Step 1 — Create and Activate a Virtual Environment

```
$ python -m venv venv
# Windows
$ venv\Scripts\activate
# macOS / Linux
$ source venv/bin/activate
```

Step 2 — Install Dependencies

```
(venv) $ pip install Flask==3.0.3
(venv) $ pip install scikit-learn==1.5.1
(venv) $ pip install pandas==2.2.2
(venv) $ pip install numpy==1.26.4
(venv) $ pip install joblib==1.4.2
(venv) $ pip freeze > requirements.txt
```

Step 3 — requirements.txt

```
Flask==3.0.3 scikit-learn==1.5.1 pandas==2.2.2
numpy==1.26.4

joblib==1.4.2
pyngrok==7.2.0
```

Step 4 — Project Directory Structure

```
hdi-predictor-dashboard/
```

```
|-- static/
```

```
| |-- css/
| |   |-- style.css
| |-- js/
| |   |-- main.js |-
- templates/
|   |-- index.html |-
- model/
|   |-- HDI.pkl
|   |-- HDI.csv |-
- app.py
|-- requirements.txt `--
- run_public.py
```

4. Project Workflow & Milestone Deck

Milestone 1: Model Serialization & Base Architecture

This milestone establishes the flat-file foundation the rest of the application depends on.

- **Activity 1.1:** Train and serialize the multi-variable regression model to model/HDI.pkl using `joblib.dump()`, and place the historical reference dataset at model/HDI.csv.
- **Activity 1.2:** Define an explicit `FEATURE_ORDER` list so that every prediction request builds its input array in the exact column order the model was trained on.
- **Activity 1.3:** Validate on startup that HDI.pkl loads without error and that HDI.csv's columns match `FEATURE_ORDER`, failing fast with a clear message if either file is missing or malformed.
- **Activity 1.4:** Define per-feature clamp boundaries (`FEATURE_BOUNDS`) used later by the Scenario Simulator to keep hypothetical inputs within a realistic envelope.

Milestone 2: Core Frontend–Backend Request Routing

This milestone defines the contract between the browser and the Flask backend before any route logic is written.

- **Activity 2.1:** Establish three JSON endpoints — `POST /predict`, `POST /simulate`, and `GET /features` — corresponding to the Predictor Engine, Scenario Simulator, and Feature Analysis Deck respectively.
- **Activity 2.2:** Standardize the response envelope across all endpoints: a top-level success boolean on the happy path, and a single error string with an appropriate HTTP status (400 for bad input, 500 for internal failures) on the failure path.
- **Activity 2.3:** Agree on the request payload shape for `/simulate` — a baseline object and an adjusted object — so the frontend can request a direct before/after comparison in a single round trip.

Milestone 3: App.py Core Logic Assembly

Milestone 3 produces the backbone of the service. The script below implements Flask routing for all three modules, a shared `validate_and_build_vector()` helper that both validates and clamps incoming indicators, and error boundaries that distinguish user input errors (400) from unexpected internal failures (500).

app.py

```
import os
import traceback

import joblib
import numpy as np
import pandas as pd
from flask import Flask, jsonify, render_template, request

app = Flask(__name__)

# -----#
# Model & reference-data loading (flat-file, in-memory – no database layer)
# -----#
```



```

MODEL_PATH = os.path.join("model", "HDI.pkl")
DATA_PATH = os.path.join("model", "HDI.csv")

model = joblib.load(MODEL_PATH) reference_df =
pd.read_csv(DATA_PATH) # Exact feature order the
regression model was trained on. FEATURE_ORDER = [
    "life_expectancy",
    "expected_years_schooling",
    "mean_years_schooling",
    "gni_per_capita",
    "education_spend_pct_gdp",
    "healthcare_access_index",
]
# Realistic clamp boundaries used by the Scenario Simulator to prevent #
physically impossible or policy-irrelevant inputs from reaching the model.
FEATURE_BOUNDS = {
    "life_expectancy": (30.0, 90.0),
    "expected_years_schooling": (0.0, 22.0),
    "mean_years_schooling": (0.0, 18.0),
    "gni_per_capita": (300.0, 150000.0),
    "education_spend_pct_gdp": (0.0, 15.0),
    "healthcare_access_index": (0.0, 100.0), }
def clamp(value: float, lo: float, hi: float) -> float:
    """Clamp a numeric value into [lo, hi], used by the Scenario Simulator
    to keep volatile hypothetical inputs within a realistic envelope."""
    return max(lo, min(hi, value))
def validate_and_build_vector(payload: dict) ->
np.ndarray:
    """Validate an incoming indicator payload and assemble it into the
    ordered feature array the model expects. Raises ValueError on any
    missing field or on a value that fails type coercion."""
    values = []
    for feature in FEATURE_ORDER:
        if feature
not in payload:
            raise ValueError(f"Missing required
indicator: {feature}")
        try:
            raw_value =
float(payload[feature])
        except (TypeError, ValueError):
            raise ValueError(f"Indicator '{feature}' must be numeric")
        lo, hi = FEATURE_BOUNDS[feature]
        values.append(clamp(raw_value, lo, hi))
    return np.array(values, dtype=float).reshape(1, -1)

@app.route("/")

```



```
def index():
    """Renders the single-page glassmorphic dashboard interface."""
    return render_template("index.html")

@app.route("/predict", methods=["POST"])
def predict():
    """Predictor Engine – accepts a single set of socioeconomic indicators
    and returns an instant HDI estimate."""
    data = request.get_json(silent=True) or {}
    try:
        feature_vector = validate_and_build_vector(data)
        prediction = float(model.predict(feature_vector)[0])
        return jsonify({
            "success": True,
            "hdi_score": round(prediction, 4),
            "inputs_used": dict(zip(FEATURE_ORDER, feature_vector.flatten().tolist())),
        })
    except ValueError as exc:
        return jsonify({"error": str(exc)}), 400
    except Exception as exc: # pragma: no cover - defensive boundary
        traceback.print_exc()
        return jsonify({"error": f"Prediction failed: {exc}"}), 500

@app.route("/simulate", methods=["POST"])
def simulate():
    """Scenario Simulator – clamps and re-runs a modified indicator set
    against a baseline so planners can compare 'what-if' policy shifts."""
    data = request.get_json(silent=True) or {}
    baseline_payload = data.get("baseline", {})
    adjusted_payload = data.get("adjusted", {})
    if not baseline_payload or not adjusted_payload:
        return jsonify({"error": "Both 'baseline' and 'adjusted' indicator sets are required."}), 400
    try:
        baseline_vector = validate_and_build_vector(baseline_payload)
        adjusted_vector = validate_and_build_vector(adjusted_payload)

        baseline_score = float(model.predict(baseline_vector)[0])
        adjusted_score = float(model.predict(adjusted_vector)[0])
        delta = adjusted_score - baseline_score
        pct_change = (delta / baseline_score * 100.0) if baseline_score else 0.0

        return jsonify({
            "success": True,
            "baseline_hdi": round(baseline_score, 4),
            "adjusted_hdi": round(adjusted_score, 4),
            "delta": round(delta, 4),
            "percent_change": round(pct_change, 2),
        })
    except Exception as exc:
        return jsonify({"error": str(exc)}), 500
```

```

    })    except ValueError as exc:
return jsonify({"error": str(exc)}), 400
    except Exception as exc: # pragma: no cover - defensive boundary
        traceback.print_exc()
        return jsonify({"error": f"Simulation failed: {exc}"}), 500

@app.route("/features", methods=["GET"])
def features():
    """Feature Analysis Deck – exposes the model's learned per-feature
    weights so the frontend can render a structural impact chart."""
    try:
if not hasattr(model, "coef_"):
    raise ValueError("Loaded model does
not expose linear coefficients.")
        weights = dict(zip(FEATURE_ORDER, model.coef_.tolist()))
return jsonify({"success": True, "weights": weights, "intercept":
float(model.intercept_)})
    except Exception as exc:
        traceback.print_exc()
        return jsonify({"error": f"Could not load feature weights: {exc}"}), 500
    if __name__ == "__main__":
app.run(debug=True, port=5050)

```

Figure 4.1 — Complete Flask application logic for the Predictor Engine, Scenario Simulator, and Feature Analysis Deck.

Milestone 4: Frontend Development Framework

- **Activity 4.1:** Build templates/index.html as a single-page interface containing an input section, a scenario-comparison panel, and a results panel, all styled with the dark glassmorphic theme in static/css/style.css.
- **Activity 4.2:** Link static assets (style.css, main.js) via Flask's `url_for('static', filename=...)` so paths remain correct regardless of deployment host.
- **Activity 4.3:** Implement main.js to intercept form submission, POST JSON to the appropriate endpoint via `fetch()`, and render the JSON response into dynamically created result cards without a full page reload.
- **Activity 4.4:** Add a lightweight client-side range check mirroring `FEATURE_BOUNDS` so users get immediate feedback before the request ever reaches the server.

Milestone 5: Local Testing & Ngrok Tunnel Deployment

- **Activity 5.1:** Run the Flask development server locally and exercise all three endpoints — a baseline prediction, a scenario comparison, and a feature-weight fetch — to confirm end-to-end wiring.

```

(venv) $ python app.py
*      Serving Flask app 'app'      *
Debug mode: on

```

```
* Running on
http://127.0.0.1:5050Press CTRL+C to
quit
```

- **Activity 5.2:** Install pyngrok and configure the authtoken node so the local server can be exposed on a public edge gateway URL for evaluator access.

```
(venv) $ pip install pyngrok
(venv) $ python
>>> from pyngrok import ngrok
>>> ngrok.set_auth_token("YOUR_NGROK_AUTHTOKEN")
```

- **Activity 5.3:** Create run_public.py to open the tunnel and start the Flask server with debug=False, avoiding the reloader process that would otherwise interfere with Ngrok's tunnel management.

```
from pyngrok import ngrok
from app import app

public_url = ngrok.connect(5050)
print("=" * 55)
print("YOUR PUBLIC WEBSITE URL IS LIVE!")
print(f"URL: {public_url}")
print("=" * 55)

app.run(port=5050, debug=False)
```

- **Activity 5.4:** Stream the live public gateway endpoint to evaluators, noting that the free-tier Ngrok URL rotates on every restart of run_public.py.

Milestone 6: Analytical Project Conclusion

The final milestone closes the delivery cycle by confirming that the Predictor Engine, Scenario Simulator, and Feature Analysis Deck all behave consistently against the historical validation rows in HDI.csv, documenting known limitations of the regression approach, and recording the roadmap items carried into the next iteration (see Section 6).

5. Full Web Application Interface Walkthrough

Home / Dashboard Layout

The landing view uses a dark glassmorphic theme: a near-black background hosts several softly blurred, slowdrifting color blobs behind translucent, frosted-glass panels with subtle border highlights. Typography leans on a clean geometric sans-serif for headings and a slightly smaller weight for body copy, keeping numeric output (the HDI score itself) visually dominant. The header carries the dashboard's name and a one-line tagline, with the three module panels — Predictor Engine, Scenario Simulator, and Feature Analysis Deck — presented as selectable tabs or stacked cards on a single scrollable page, consistent with the project's flat, nologin, no-database design philosophy.

Metric Input Section

Each socioeconomic indicator is exposed as a labeled numeric field with an inline range hint (for example, 'Life expectancy at birth, years — typical range 30–90') so users understand the clamp boundaries before

submitting. Placeholder text cycles through realistic example values drawn from HDI.csv so a first-time user can see plausible magnitudes at a glance. Fields enforce basic type checking client-side, and the primary action button transitions into a subtle spinner animation the moment a request is in flight, disabling further submissions until the backend responds.

Results Analytics Section

On a successful response, the results panel injects a primary result card showing the predicted HDI score to four decimal places, plus, for Scenario Simulator runs, a secondary comparison card showing the baseline score, the adjusted score, the absolute delta, and the percentage change, color-coded green or amber depending on direction. Numeric values include a copy-to-clipboard control for quick citation in reports. Once the cards are populated, the page smooth-scrolls the viewport down to the results section automatically so the user does not have to hunt for the newly rendered output, and the Feature Analysis Deck renders a horizontal bar chart of the model's per-feature coefficients pulled from the /features endpoint, giving an at-a-glance view of which indicators the model weighs most heavily.

6. Conclusion & Platform Roadmap

Summary

The Global Well-Being & HDI Predictor Dashboard demonstrates that a meaningfully useful predictive analytics tool for development policy work does not require a heavyweight server stack. By serializing a trained regression model to a flat binary file, keeping the historical validation dataset as a simple CSV, and exposing three narrowly scoped Flask endpoints, the project delivers a genuinely zero-server-footprint analytics workload: nothing but Python, a model file, and a browser is required to go from raw socioeconomic indicators to a defensible HDI estimate, a policy 'what-if' comparison, and a transparent view of which factors the model considers most influential.

Strategic Roadmap

- **1. Low-Footprint Historical Query Tracking:** Introduce a lightweight embedded store (e.g., SQLite) purely for logging past prediction and simulation requests, enabling trend review across sessions without adopting a full server-based database.
- **2. Multi-Language UI Localization Matrix:** Externalize all interface strings into locale files so the dashboard can be presented in additional languages for international development audiences without touching application logic.
- **3. Advanced Time-Series Forecasting Integration:** Extend the Predictor Engine with a time-aware model (e.g., a gradient-boosted or recurrent architecture trained on multi-year indicator trends) to project HDI trajectories forward rather than producing a single point-in-time estimate.